System Design

Topic that will cover:

- Architectural Design Design
- Principles: Coupling & Cohesion
- Data Flow Diagrams (DFDs)
- Entity Relationship Diagrams (ERDs)
- UML Diagrams (Class, Sequence, Activity)

What is system design?

System Design is the process of **planning the architecture and components of a software system**. It helps answer **"how"** a system will be built after we know **"what"** it should do (from requirements analysis).

Architectural Design

What is it?

- Architectural Design defines the **overall structure of a software system** how different modules (parts) work together.
- A software system needs **architecture** to show how **different parts (modules)** will work together.
- **Architectural Design** is the **blueprint** of a software system.
- It is done **after requirement analysis** and before detailed design or coding.

Key Concepts:

- **Layers or Tiers**: Presentation layer (UI), Business Logic layer, Data layer.
- **Client-Server Architecture**: Like websites – user (client) interacts with a server.
- **Microservices**: System split into small services – each handling a specific task (e.g., login service, payment service).

Why is it Important?

- Helps break the system into smaller, manageable parts.
- Allows teams to work on different components.
- Improves **scalability**, **security**, **performance**, and **maintainability**.
- Prevents future problems by thinking through structure early.

1. Layers or Tiers (Layered Architecture)

What is it?

- Software is divided into different **layers**, and each layer has its own responsibility.

Common 3-Tier Layers:

Layer	Description	Example
Presentation Layer (UI)	The front-end the user interacts with	Mobile app screen, website
Business Logic Layer	Contains logic/rules of the system	"If password correct → login", "If steps ≥ 10,000 → show reward"
Data Layer	Handles database operations	Reading/writing from SQLite, Firebase

Flow:

User clicks button (UI) → Logic is processed (Business Layer) → Data is fetched/saved (Data Layer) → Response sent back

Benefits:

- Easier to update one layer without touching others.
- Promotes **separation of concerns**.

2. **Client-Server Architecture**

What is it?

A model where:

- **Client**: Requests services (e.g., a browser, mobile app).
- **Server**: Responds to those requests (e.g., a backend server).

Example:

In a food delivery app:

- **Client**: The app on your phone
- **Server**: The backend that handles login, orders, and payments.

Client sends: "I want to order pizza"
Server replies: "Pizza ordered and arriving in 30 minutes."

Benefits:

- Centralized system – easy to manage and secure.
- Can serve multiple clients at once.

3. Microservices Architecture

What is it?

- System is broken into **small, independent services**, and each handles one **specific function**.

Each service:

- Has its own logic.
- Can use a separate database.
- Communicates with other services using APIs.

Example in a Fitness App:

Microservice	**Task**
Login Service	Handles user login and signup
Step Counter Service	Tracks steps and stores them
Nutrition Service	Manages meal data
Notification Service	Sends water/drink reminders

- Each service can be built, deployed, and updated **independently**.

Benefits:

- **Scalable** – You can scale only the step counter if needed.
- **Flexible** – Can use different languages/technologies for each.
- **Reliable** – If one service fails, the whole app doesn't crash.

4. Monolithic Architecture

What is it?

- All features and logic are built into **a single unit** (one app or codebase).

Example:

- Old apps where UI, logic, and database were in one big .exe or .jar.

Advantages:

- Simple to develop and deploy (at first)
- Works well for small apps

Disadvantages:

- Hard to scale or modify
- One bug can break the whole app

Other Architectural Styles:

Style	Description	Example
Monolithic	All parts in one big unit	Traditional apps, hard to scale
Event-Driven	System reacts to events (button click, message received)	Used in UI-heavy or IoT systems
Service-Oriented Architecture (SOA)	Like Microservices but with shared services & enterprise-wide scope	Enterprise apps
MVC (Model-View-Controller)	Separates data (Model), UI (View), and logic (Controller)	Used in web frameworks like Django, Laravel

What is an Architectural Style?

An **architectural style** is a **standard design pattern** or **template** used to structure a software system.

It defines:

- **How components interact**
- **How data flows**
- **How responsibilities are distributed**

✓ Think of it like a building plan — just like houses can be bungalows, apartments, or villas, software systems can be built using different architectural styles.

Architectural Style vs Architectural Design:

Aspect	Architectural Style	Architectural Design
Definition	A **general pattern** or **template** for organizing a system.	The **actual plan** of your system based on selected styles.
What it is like	Like choosing a **building style**: bungalow, apartment, villa.	Like drawing the **floor plan** of your specific bungalow or villa.
Focus	Focuses on **common structures and principles** used in many systems.	Focuses on the **structure of a specific system** (your project).
Examples	- Layered- Client-Server- Microservices- MVC- Event-Driven	- Your system has 3 layers: 1. UI for mobile 2. Business logic in Java 3. SQLite database
Purpose	Provides a **template or guideline** for design.	Provides the **real design and structure** of your system.

Building a **Fitness Tracker App**.

- **Architectural Style**: You decide to use the **Layered Architecture** style.
- **Architectural Design**: You create:
    - A **UI Layer** with XML/Jetpack Compose.
    - A **Logic Layer** with Java/Kotlin to process steps.
    - A **Data Layer** with SQLite to store steps and goals.

Design Principles: Coupling & Cohesion:

What Are Design Principles?

- In software design, **principles** are like **rules or good habits** to write clean, reliable, and easy-to-maintain code.
- Two most important principles are:
    a. **Coupling** – how much one part of your code depends on another.
    b. **Cohesion** – how focused and clear each part of your code is.

Coupling (Connection Between Modules)

- **Definition**: How **dependent** one module is on another.
- **Types**:
    - **Tight Coupling (Bad)**: Modules are highly dependent.
    - **Loose Coupling (Good)**: Modules can work independently.

Example: If changing one module breaks another, it's tightly coupled.

Think of It Like:

Imagine two friends working together. If one can't do anything without asking the other every time → they are **tightly coupled**.

Loose Coupling (Good):

- Modules **communicate with each other** as little as possible.
- One module can change **without breaking** others.
- Easier to **test**, **maintain**, and **re-use**.

Example:

- A LoginModule that just calls a UserDatabaseModule through an interface:
- User user = userDatabase.getUser(email);
- Now if the database changes (e.g., from SQLite to Firebase), LoginModule won't break.

Tight Coupling (Bad):

- Modules are **heavily connected** or rely too much on each other.
- A small change in one place can **crash** another module.
- Hard to update, test, or reuse.

Example:
If your LoginModule directly accesses the database with SQL queries and shares variables with EmailModule, any small change may affect multiple modules.

Cohesion (How Clear a Module Is Internally))

- **Definition**: How **focused** a module is on one task.
- **Types**:
    - **Low Cohesion (Bad)**: Module does many unrelated tasks.
    - **High Cohesion (Good)**: Module does one task well.

Example: A class that only handles user login is cohesive. If it also sends emails and saves reports that's low cohesion.

Think of It Like:

- If a student focuses only on studying, that's **high cohesion**.
- But if the student is also cleaning rooms, cooking, and managing events – that's **low cohesion** (too many unrelated tasks).

High Cohesion (Good):

- A module does **one thing only** – and does it well.
- Easy to **understand**, **update**, and **re-use**.

Example:
A ReportGenerator class that:

- Takes data
- Formats it
- Generates a report

That's high cohesion – all actions are related to **reporting**.

Low Cohesion (Bad):

- A module does **multiple unrelated tasks**.
- Confusing and difficult to manage.

Example:
A UserManager class that:

- Handles login
- Sends email
- Calculates BMI
- Schedules water reminders

Now this class is doing **too many jobs** that's low cohesion.

Building a Fitness Tracker App:

➤ High Cohesion, Loose Coupling:

- StepTrackerModule: only tracks steps.
- CalorieCalculatorModule: only calculates calories.
- They talk to each other through **clear interfaces**.

✓ Easy to update the step tracker without breaking calorie calculator.

➤ Low Cohesion, Tight Coupling:

- One class does **step tracking + notifications + saving to DB + calorie burn logic**
- All modules **share internal data** and call each other's private functions.

✓ Change one thing, and everything else breaks.

Data Flow Diagrams (DFDs):

What is a DFD?

A Data Flow Diagram (DFD) is a **graphical tool** that shows:

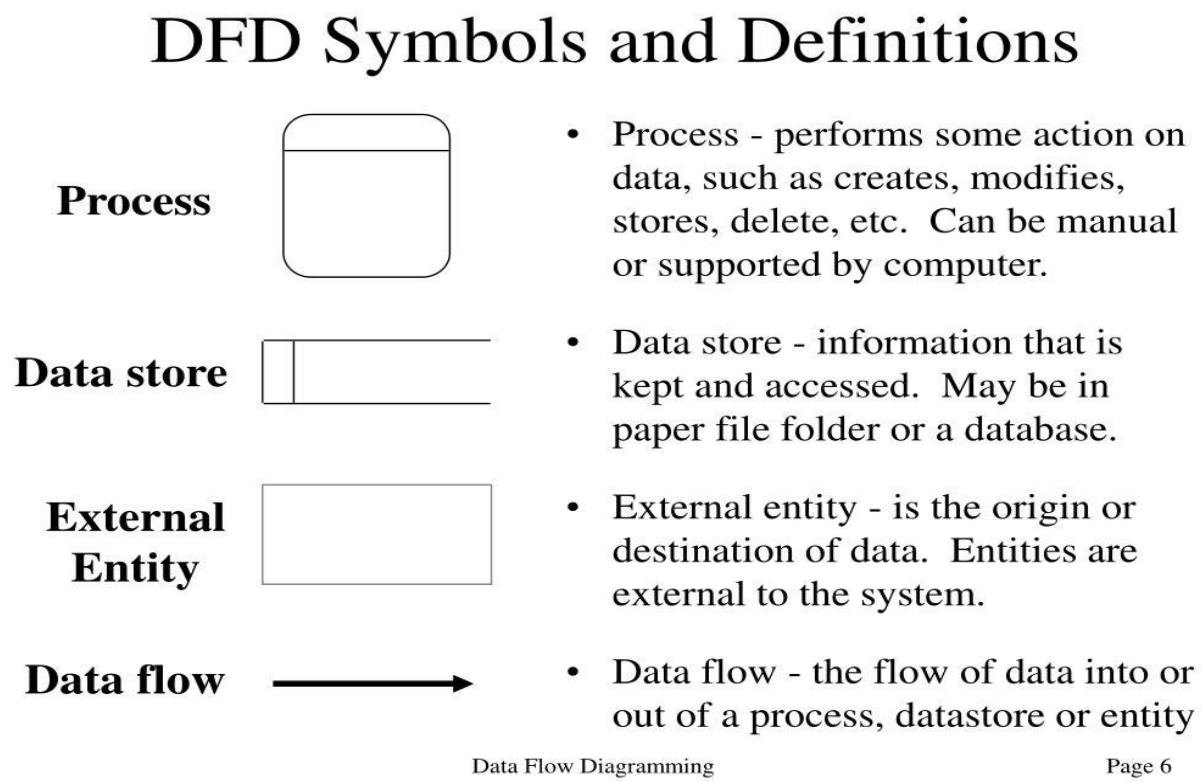
- **How data moves** through a system
- **Where data is stored**
- **What processes transform the data**

➢ It focuses on the **flow of data**, **not the code** or technical implementation.

Why Use DFDs?

- To **understand** the system without needing to know programming.
- To **communicate clearly** between developers, clients, and users.
- To **analyze** and **design** the system before building it.

Key Elements of a DFD



1. External Entity (Square/Rectangle)

- Represents **users** or **systems outside** the system you're designing.
- They provide **input** or receive **output**.

Example: User, Admin, Bank, Payment Gateway

2. Process (Circle or Rounded Rectangle)

- Represents a function that **transforms data** (input → output).
- A **process takes in data**, performs logic, and **outputs** new data.
-

Example: "Login", "Calculate BMI", "Verify Payment"

3. Data Store (Open Rectangle or Two Lines)

- A **place where data is stored** for later use.
- It can be a database, a file, or memory.

Example: User_DB, Step_Records, Orders_List

4. Data Flow (Arrow)

- Shows **data movement** between:
  - Entities ↔ Processes
  - Processes ↔ Data Stores
  - Processes ↔ Processes

Example: "Login info", "Calories data", "User ID"

DFD Levels:

Level 0 – Context Diagram

- The **top-level** DFD.
- Shows the **entire system as one process (circle)**.
- Includes only **external entities** and **data flows**.

Example: Fitness App

- One big process: FitTrack System
- External entities: User, Cloud Server
- Arrows: User data → FitTrack System → Step Count Info

Level 1 DFD

- Breaks down the single process into **main sub-processes**.
- More detailed than Level 0.
- Shows **major functions** like "Login", "Track Steps", "Store Data".

Example:

- Processes:
  - 1.0 Login
  - 2.0 Track Steps
  - 3.0 View History
- Data stores:
  - User_DB
  - Steps_DB

Level 2 DFD

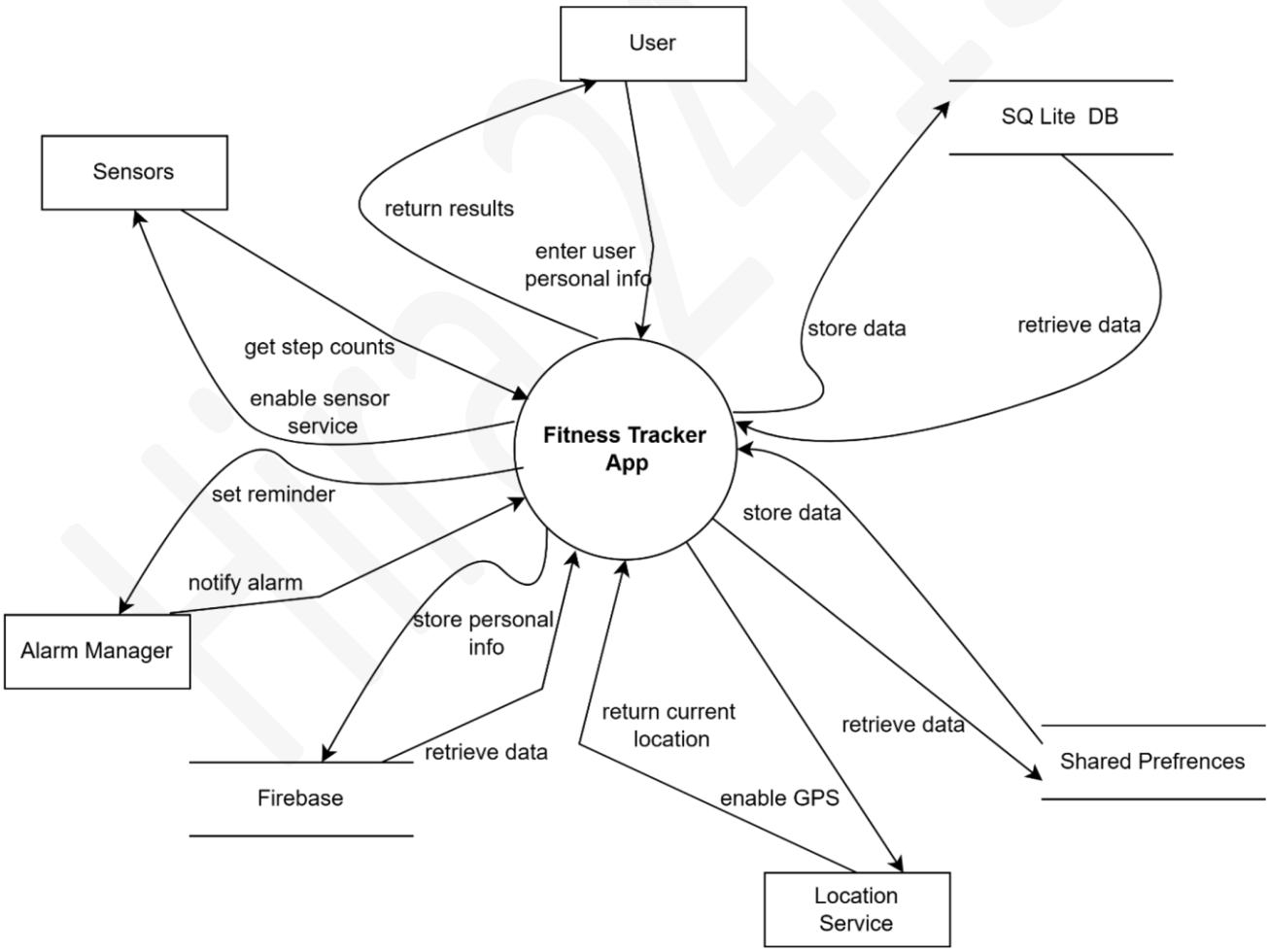
- Breaks down **each Level 1 process** into **more detailed sub-processes**.
- Shows **specific logic and smaller steps**.

Example:

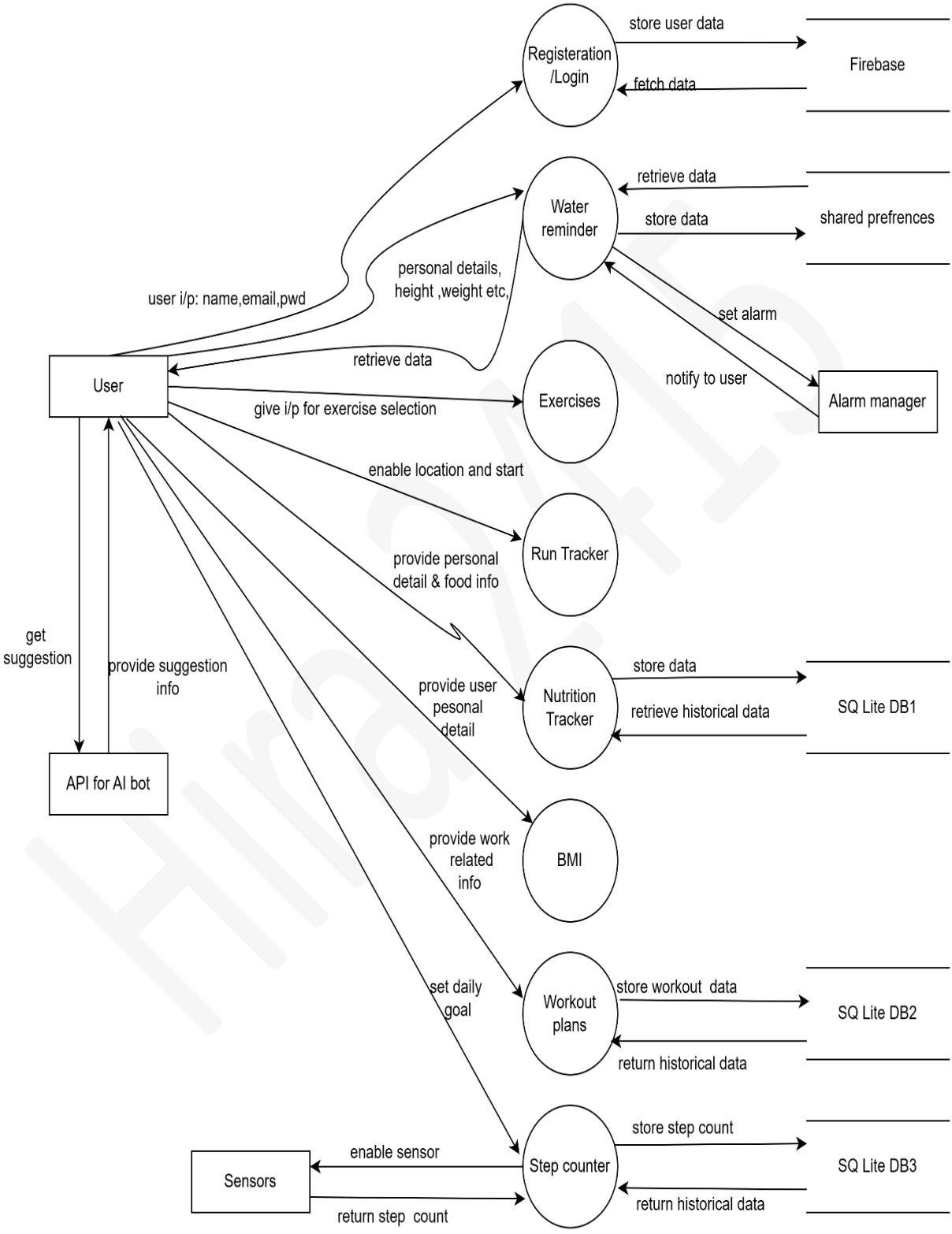
- Inside 2.0 Track Steps:
    - 2.1 Detect Steps
    - 2.2 Update Total
    - 2.3 Save to DB

Benefits of DFDs:

- **Visual** and **easy to understand**
- Helps catch **errors in design**
- **Separates data and logic**
- Useful in both small and large projects



Level 0 DFD



Level 1 DFD

Entity Relationship Diagrams (ERDs)

What is an ERD?

An ERD shows the **relationships between entities (tables)** in a database.

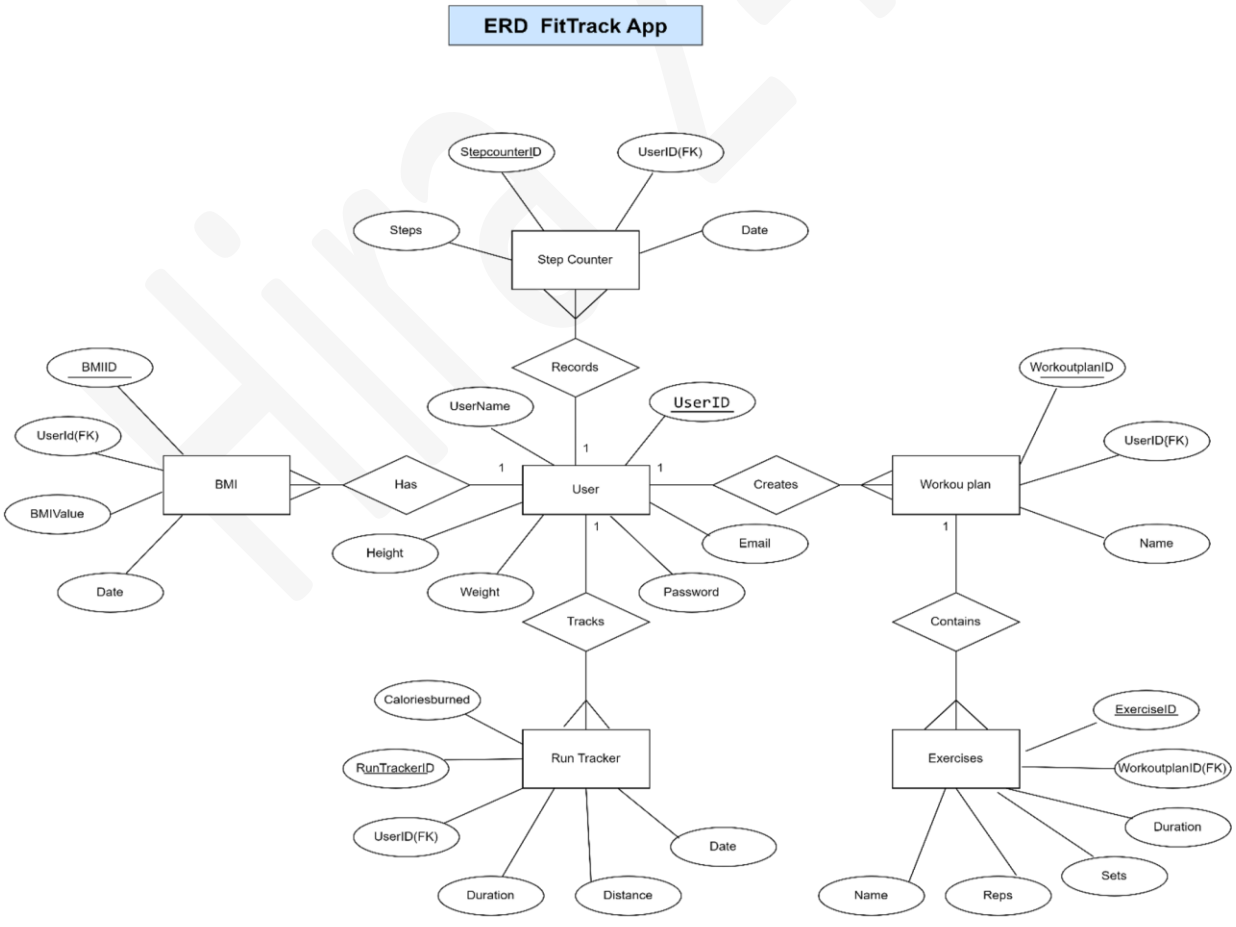
Elements:

- **Entity**: A real-world object (e.g., Student, Course) – shown as a rectangle.
- **Attribute**: A property (e.g., student_name) – shown as ovals.
- **Relationship**: How entities are connected (e.g., "enrolls in") – shown as a diamond.

Types of Relationships:

- **One-to-One**
- **One-to-Many**
- **Many-to-Many**

Example: A Student *enrolls in* many Courses. A Course can have many Students.

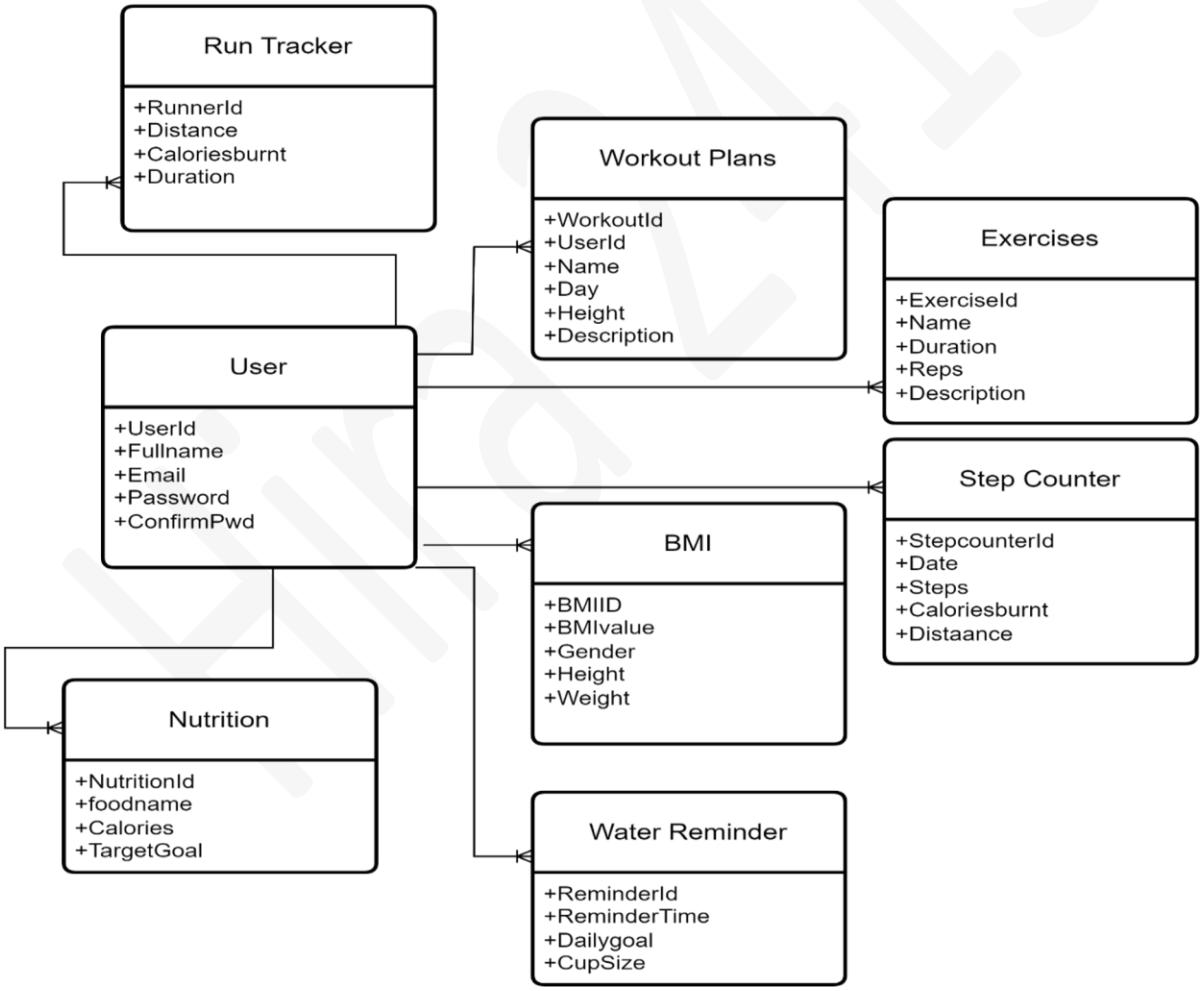


Domain Model

The domain model represents the conceptual structure of the system, identifying key entities and their relationships.

Entities and Relationships:

- **User**: Tracks personal data such as name, age, gender, and goals.
- **Step Counter**: Logs steps, distance, and calories.
- **Nutrition Tracker**: Records meals and macronutrients.
- **Water Reminder**: Manages water intake schedules.
- **BMI Calculator**: Calculates BMI based on user input.
- **Run Tracker**: Tracks running activities using GPS.
- Relationships are established using associations (e.g., a User "logs" steps or "tracks" nutrition).



Purpose of a Domain Model

- Understand the **core concepts** of your system
- Show the **relationships** between entities
- Communicate with non-technical people (like clients)
- Prepare for database and class design

Features of a Good Domain Model

- Shows **real-world objects**
- Includes only important attributes and relationships
- **No technical details** like SQL, UI, APIs – only **concepts**

UML Diagrams (Unified Modeling Language)

- UML is a **standard way to visualize the design** of an object-oriented system.

1. Class Diagram

Shows the **classes in a system and their relationships**.

Includes:

- Class name
- Attributes (variables)
- Methods (functions)
- Relationships like:
    - Inheritance (is-a)
    - Association (uses)
    - Aggregation (has-a)

Example: Student class has name, roll_no, and a method registerCourse().

2. Sequence Diagram

- Shows the **order of messages** between objects **over time**.

Includes:

- Objects (shown at the top)
- Lifelines (vertical lines)
- Messages (arrows) – function calls or data transfer

Example: User → clicks login → system → checks password → returns success.

3. Activity Diagram

- Shows the **flow of actions (like a flowchart)** in a system or process.

Includes:

- Start node
- Actions (rounded rectangles)
- Decisions (diamonds)
- End node

Example: Start → Enter username → Check validity → If correct → Proceed → End.

Note: These UML Diagrams will be read in next semester 5th in OOAD Course.